



IDEAL Product and Technical Specification

Trusted Digital Deals and Contracts

Project	IDEAL
Version	1.0
Status	Draft for Team Alignment
Date	June 2026
Owner	Project Lead

Official product, architecture, backend, mobile, admin, database, QA, and delivery specification.

Contents

1	Executive Summary	6
2	Project Charter	7
2.1	Vision	7
2.2	Problem Statement	7
2.3	Goals	7
2.4	Non-Goals	7
2.5	Success Criteria	7
2.6	Ownership	7
3	Product Requirements Document	9
3.1	Product Overview	9
3.2	Target Users	9
3.3	Core Capabilities	9
3.4	Constraints and Dependencies	9
3.5	Product Acceptance Criteria	9
4	Software Requirements Specification	10
4.1	Requirement Catalog	10
4.2	Acceptance Criteria	10
5	MVP Scope Statement	11
5.1	Included	11
5.2	Excluded	11
5.3	Definition of Done	11
6	Stakeholders and User Roles	12
7	Business Rules Specification	13
7.1	Canonical Deal Lifecycle	13
7.2	Rules	13
7.3	State Transition Table	14
7.4	Approval and Locking Rule	14
8	User Journey and Workflow Specification	16
8.1	Core Flow Pattern	16
8.2	Deal Approval Flow	16
8.3	Failure Cases	16
8.4	Detailed Workflow Matrix	16

9	UI/UX Specification	18
9.1	Mobile Route Catalog	18
9.2	State Rules	18
10	System Architecture Specification	19
10.1	Layer Responsibilities	19
10.2	System Context Diagram	19
10.3	Authentication Flow	20
11	Backend Architecture Specification	21
11.1	Planned Structure	21
11.2	Patterns	21
11.3	Backend Module Catalog	21
11.4	DTO and Guard Catalog	22
11.5	Standard API Response and Error Format	23
12	Data Model Specification	24
12.1	Core Tables	24
12.2	Relationships	24
12.3	Implementation Schema	24
12.4	Migration Plan	25
13	Security and Access Control Specification	26
13.1	Rules	26
13.2	Access Controls	26
13.3	RLS and Storage Policy Specification	26
13.4	Environment Variable Catalog	27
14	API and Integration Specification	28
14.1	Base Path	28
14.2	Endpoint Examples	28
14.3	Implementation API Catalog	28
14.4	Request and Response Examples	29
15	Admin Dashboard Specification	31
15.1	Admin Route Catalog	31
15.2	KYC Review Workflow	32
15.3	Permission Matrix	32
16	Mobile Application Specification	33
16.1	Important Refactor Note	33
16.2	Current Implementation Status	33
16.3	Target Screen Specification	33
17	File Storage and Media Specification	35
18	Notifications and Realtime Specification	36
19	Payments and Subscription Specification	37
20	Identity Verification Specification	38

21 Quality Assurance and Testing Plan	39
21.1 Required Scenarios	39
21.2 Testing Matrix	39
21.3 CI Checks	40
22 Development Workflow Guidelines	41
22.1 Branch Naming	41
22.2 Commit Format	41
22.3 Daily Update	41
23 Delivery Plan and Milestones	42
23.1 Deliverables by Owner	42
23.2 Deployment Environments	43
24 Risk Register	44
25 Decision Log and Open Issues	45
25.1 Current Decisions	45
25.2 Open Issues	45
25.3 Document Control	45
Backend Implementation Contract	46
Admin Dashboard Specification	51
Mobile Integration Contract	54
QA Acceptance Test Matrix	57
Implementation Status Checklist	61
A Glossary	64
B Status Codes and Enums	65
B.1 Deal Status	65
B.2 KYC Status	65
B.3 Subscription Status	65
B.4 Additional Enums	65
C Database Table Catalog	66
C.1 Column-Level Implementation Notes	66
D API Endpoint Catalog	67
E Test Scenarios	68
F Revision History	69

List of Tables

25.2 Backend common components	47
25.3 Backend module implementation contract	47
25.4 Admin roles and permissions	52
25.5 Admin route catalog	52
25.6 Admin component catalog	53
25.7 Mobile screen specification	55
25.8 QA acceptance matrix	57
25.9 Implementation status checklist	61
F.1 Document control and revision history	69

List of Figures

10.1 IDEAL target architecture data flow	19
10.2 Authentication and authorization flow	20

Chapter 1

Executive Summary

IDEAL is a cross-platform platform for creating, negotiating, approving, versioning, and managing trusted digital deals and contracts between individuals, companies, and multiple involved parties.

The product exists to reduce ambiguity in informal agreements by giving users a clear workflow: create a deal, invite parties, negotiate terms, approve the final version, lock the official version, and manage future changes through new approved versions.

Target users include registered individuals, verified users, deal creators, invited parties, company owners, company members, administrators, and system service accounts. The target architecture uses Flutter for mobile presentation, Next.js for administration, NestJS for business workflows, Supabase PostgreSQL for persistence, Supabase Auth for identity sessions, Supabase Storage for private files, Supabase Realtime for updates, Stripe for subscriptions, and an external provider for KYC.

Important Note

Flutter and Next.js must not own sensitive business logic. Deal lifecycle, approval, version locking, trust counters, subscriptions, audit logs, KYC decisions, admin actions, and protected file access must be enforced through the NestJS API.

The MVP objective is to deliver the foundation for authentication, profile completion, deal creation, party invitation, deal versioning, approval, rejection, change requests, locked approved versions, private file attachments, notifications, audit logging, and basic admin review.

Chapter 2

Project Charter

2.1 Vision

IDEAL will become the trusted workflow layer for digital deals and contracts, with transparent approvals and immutable approved versions.

2.2 Problem Statement

Users often negotiate agreements through fragmented messages, documents, and verbal confirmation. The result is weak traceability, unclear approvals, and unsafe post-approval editing.

2.3 Goals

- Provide a simple deal workflow for individuals and companies.
- Lock approved contract versions and preserve historical versions.
- Centralize sensitive workflows inside the backend API.
- Give admins audited review and moderation tools.

2.4 Non-Goals

The MVP does not include AI drafting, legal template marketplaces, advanced analytics, enterprise contract lifecycle management, or automated multi-country compliance.

2.5 Success Criteria

The MVP is successful when users can create, invite, negotiate, approve, lock, and revise deals with backend-enforced access control and audit logs.

2.6 Ownership

Role	Ownership
Mobile owner	Flutter application and presentation workflows.

Backend owner	NestJS API, business logic, integrations, guards, and audit triggers.
Database owner	Supabase schema, migrations, RLS, storage policies, and retention.
Documentation owner	Product and technical specifications.
Project lead	Final decisions, scope approval, and prioritization.

Chapter 3

Product Requirements Document

3.1 Product Overview

IDEAL enables trusted digital deal management from draft to locked approval.

3.2 Target Users

Guests, registered users, verified users, deal creators, invited parties, company owners, company members, admins, super admins, and system service accounts.

3.3 Core Capabilities

- User account, profile, identity verification, and trust counters.
- Deal creation, invitation, negotiation, versioning, approval, rejection, and locking.
- Deal-scoped chat, private file attachments, notifications, subscriptions, and admin moderation.

3.4 Constraints and Dependencies

The backend must be the source of truth for business rules. Stripe, Supabase, and the KYC provider are external dependencies. Frontend applications may display state but must not directly mutate protected business state.

3.5 Product Acceptance Criteria

Users can complete the MVP deal lifecycle, unauthorized users cannot access protected deals or files, locked versions cannot be edited, and every sensitive action is auditable.

Chapter 4

Software Requirements Specification

4.1 Requirement Catalog

ID	Title	Description	Priority	Owner
FR-001	Create Deal	Authenticated users can create draft deals.	High	API
FR-002	Approve Version	Required parties can approve the same submitted version.	High	API
FR-003	Lock Version	A fully approved version becomes official and immutable.	High	API
FR-004	Signed Files	File uploads and downloads use backend-authorized signed URLs.	High	API
NFR-001	Maintainability	Modules follow clear ownership boundaries.	High	All
NFR-002	Observability	Sensitive workflows expose logs and audit records.	Medium	API
SEC-001	Auth Enforcement	Protected endpoints validate JWT sessions.	High	API
SEC-002	Resource Access	Users only access deals where they are creator or participant.	High	API
AUD-001	Audit Actions	Sensitive actions create immutable audit logs.	High	API
AUD-002	Admin Actions	Admin decisions are attributable and reviewable.	High	API

4.2 Acceptance Criteria

Each requirement is accepted when it has automated coverage where practical, documented API behavior, validation rules, error cases, and ownership assigned.

Chapter 5

MVP Scope Statement

5.1 Included

The MVP includes Flutter foundation, authentication flow, user profile, deal creation, deal list, deal details, party invitation, party viewing, initial deal version creation, submission, approval, rejection, change requests, version locking, post-approval new versions, basic notifications, private file attachment support, audit logging, and basic admin review.

5.2 Excluded

Advanced AI drafting, legal template marketplaces, advanced analytics, public marketplaces, complex organization billing, advanced dispute resolution, and final production KYC integration are excluded.

5.3 Definition of Done

The MVP is done when core user journeys work through the NestJS API, protected state cannot be bypassed from clients, and QA scenarios for locking, authorization, files, subscriptions, and audit logs pass.

Chapter 6

Stakeholders and User Roles

Role	Permissions	Restrictions	Backend Rule
Guest	View public entry points.	No protected data access.	Anonymous only.
Registered user	Manage own profile and create deals.	No verified-only actions if KYC required.	JWT required.
Verified user	Use trust-sensitive workflows.	Cannot approve for others.	KYC status checked.
Deal creator	Manage drafts, invite parties, submit versions.	Cannot edit locked versions.	Creator or participant check.
Invited party	Review, approve, reject, request changes.	Cannot manage unrelated parties.	Party membership check.
Company owner	Manage company settings and members.	Cannot bypass deal rules.	Company RBAC.
Company member	Act within assigned company permissions.	Limited by role.	Company RBAC.
Admin	Review users, KYC, reports, deals, audits.	All actions audited.	Admin guard.
Super admin	Manage platform admins and settings.	Highest accountability.	Super-admin guard.
Service account	Run backend integrations.	No interactive login.	Scoped service credentials.

Chapter 7

Business Rules Specification

7.1 Canonical Deal Lifecycle

draft → negotiation → pending_approval → approved → locked

Other valid states are changes_requested, rejected, cancelled, and archived.

7.2 Rules

ID	Rule
BR-001	A deal may have many versions.
BR-002	Only one version can be the current active version.
BR-003	Only an approved version can become locked.
BR-004	A locked version cannot be edited.
BR-005	Any future change after approval creates a new version.
BR-006	A new version is not valid until all required parties approve it again.
BR-007	Users can only access deals where they are creator or participant.
BR-008	Trust counters are updated only by backend logic.
BR-009	Trial limits and paid access are enforced by the backend.
BR-010	Every important action must be written to audit logs.
BR-011	A party can be required or optional; required parties block final approval until they decide.
BR-012	A party invitation can be accepted, declined, expired, or revoked.
BR-013	Approval is valid only for the exact version that was reviewed.
BR-014	Creating a new version invalidates pending approvals on prior non-locked versions.
BR-015	A rejection returns the version to rejected state and requires a new version or explicit resubmission policy.
BR-016	A change request returns the deal to negotiation and records the requested changes.
BR-017	The prior locked version remains historically valid until a new version is fully approved and locked.
BR-018	Company deals require the actor to have company permission and deal permission.

BR-019	Chat messages are visible only to active deal participants and authorized admins.
BR-020	KYC documents are not visible to normal deal participants.
BR-021	Subscription downgrade cannot unlock features already completed, but can block new deal creation.
BR-022	Admin actions cannot directly change trust counters without a backend trust event.
BR-023	Notifications are derived from backend events, not client-side assumptions.
BR-024	Audit logs are append-only and cannot be edited through application workflows.

7.3 State Transition Table

From	To	Actor	Rule
draft	negotiation	creator	At least one invited party begins review or negotiation.
draft	pending_approval	creator	Current version has required terms and parties.
negotiation	pending_approval	creator or authorized editor	Version is submitted for approval.
pending_approval	changes_requested	participant	Change request reason is required.
pending_approval	rejected	required party	Rejection reason is required.
pending_approval	locked	backend	All required parties approved the same version.
changes_requested	negotiation	creator or editor	A new or updated draft version is prepared.
rejected	negotiation	creator or editor	A new version is created after rejection.
locked	negotiation	creator or authorized editor	Post-approval change creates a new active version while prior locked version remains historical.
draft/negotiation	cancelled	creator	Cancellation allowed before final lock subject to audit.
cancelled	archived	creator or admin	Archived for retention.
locked	archived	creator or admin	Historical record retained and hidden from active list.

7.4 Approval and Locking Rule

For IDEAL MVP, **locked** is the official final state of a version after all required parties approve the same submitted version. The deal may expose **approved** as an event or transient status, but

the implementation must treat the locked version as the immutable source of truth.

Chapter 8

User Journey and Workflow Specification

8.1 Core Flow Pattern

Each flow defines actors, preconditions, trigger, user steps, backend actions, database changes, failure cases, and success result.

8.2 Deal Approval Flow

1. Creator creates a draft deal.
2. Creator invites one or more required parties.
3. Parties review the current version and request changes or approve.
4. Backend records approvals per party and version.
5. When every required party approves the same version, backend marks the version approved and locked.

8.3 Failure Cases

Expired invitations, unauthorized deal access, stale version approval, locked-version mutation, subscription limit violation, and missing KYC eligibility must return explicit API errors.

8.4 Detailed Workflow Matrix

Flow	Actors	Backend Actions	Database Changes
Registration	guest, auth service	Create auth user, initialize profile readiness.	profiles row, audit log.
KYC submission	registered user	Validate ownership, issue signed upload URL, submit review record.	kyc_submissions, deal_files for documents, audit log.
Create deal	creator	Validate subscription, create draft aggregate.	deals row, audit log, notification optional.

Invite party	creator, invited party	Validate creator access, create invitation, notify invited user.	deal_parties, notifications, audit log.
Submit version	creator/editor	Validate active version, mark pending approval.	deal_versions status, deal status, audit log.
Approve version	required party	Validate party membership, record decision, lock if complete.	deal_approvals, deal_versions locked_at, audit log.
Request changes	participant	Record reason, return to negotiation.	approval record, version status, notification, audit log.
Upload file	participant	Authorize bucket and resource, issue signed URL.	deal_files metadata, audit log.
Admin KYC re-view	admin	Validate admin role, record decision, notify user.	kyc_submissions status, admin_actions, audit log.

Chapter 9

UI/UX Specification

9.1 Mobile Route Catalog

Screen	Route Name	Path	API Dependency
Splash	splash	/	Health/session
Welcome	welcome	/welcome	None
Login	login	/login	Auth
Register	register	/register	Auth
Forgot password	forgot-password	/forgot-password	Auth
Home dashboard	home	/home	Deals, notifications
Deals list	deals	/deals	GET /deals
Deal details	deal-details	/deals/:id	GET /deals/:id
Create deal	create-deal	/deals/new	POST /deals
Approval	approval	/versions/:id/approval	Approvals
Files	files	/deals/:id/files	Signed file URLs
Profile	profile	/profile	Profiles
KYC	kyc	/kyc	KYC
Subscription	subscription	/subscription	Subscriptions

9.2 State Rules

Screens must include loading, empty, error, and permission-denied states. Protected screens must rely on API authorization responses rather than local assumptions.

Chapter 10

System Architecture Specification

10.1 Layer Responsibilities

- Flutter owns mobile presentation and local UI state.
- Next.js owns internal admin presentation.
- NestJS owns business workflows, authorization, integrations, and audit triggers.
- Supabase PostgreSQL owns persistence.
- Supabase Auth owns authentication session lifecycle.
- Supabase Storage owns private file bytes.
- Stripe owns payment checkout and subscription events.

Decision

Backend owns business logic; frontend owns presentation; database owns persistence; storage is private by default; all sensitive actions are audited.

10.2 System Context Diagram

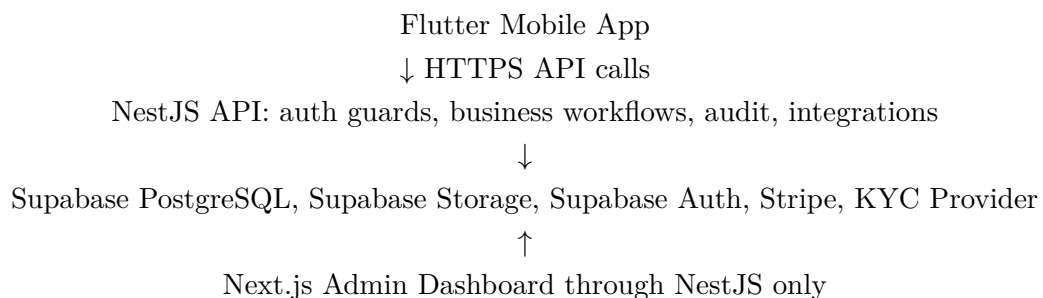


Figure 10.1: IDEAL target architecture data flow

10.3 Authentication Flow

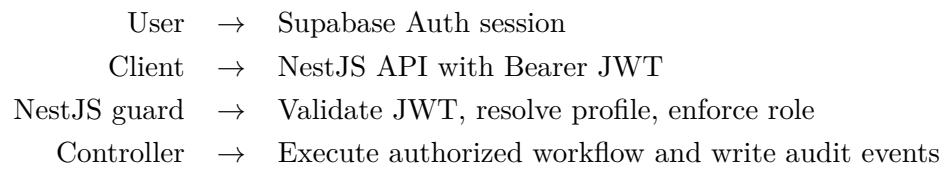


Figure 10.2: Authentication and authorization flow

Chapter 11

Backend Architecture Specification

11.1 Planned Structure

```
services/api/src/  
  auth/ users/ profiles/ kyc/ deals/ deal-parties/  
  deal-versions/ approvals/ files/ chat/ notifications/  
  trust/ subscriptions/ admin/ audit/ health/
```

11.2 Patterns

Controllers expose HTTP endpoints. Services contain business logic. Repositories or Prisma handle persistence. DTOs validate input. Guards enforce authentication, RBAC, and resource access. Filters standardize errors. Interceptors handle response and logging concerns.

Important Note

Use Prisma ORM with Supabase PostgreSQL unless another ORM decision is introduced and documented before implementation.

11.3 Backend Module Catalog

Module	Controllers and Services	Main Responsibility
auth	AuthGuard, JwtStrategy	Supabase JWT validation and profile resolution.
profiles	ProfilesController, ProfilesService	User profile read/write and completion rules.
kyc	KycController, KycService	KYC submission, status changes, document privacy.
deals	DealsController, DealsService	Deal aggregate, ownership, life-cycle transitions.
deal-parties	DealPartiesController, DealPartiesService	Party invitations, required parties, participant access.
deal-versions	DealVersionsController, DealVersionsService	Draft terms, version creation, current active version.

approvals	ApprovalsController, ApprovalsService	Approve, reject, request changes, lock approved version.
files	FilesController, FilesService	Signed URLs, metadata, private access checks.
chat	MessagesController, ChatService	Deal-scoped messages and participant-only access.
notifications	NotificationsController, NotificationsService	Notification creation, read state, realtime events.
trust	TrustService	Backend-only trust counter updates.
subscriptions	SubscriptionsController, StripeWebhookController	Trial limits, checkout, webhooks, entitlement checks.
admin	AdminController, AdminService	Internal review actions and admin permission enforcement.
audit	AuditService	Immutable audit log creation for sensitive workflows.
configuration	ConfigurationController	Employee-facing API configuration status.

11.4 DTO and Guard Catalog

Type	Name	Validation or Authorization Rule
DTO	CreateDealDto	title required, description optional, currency ISO code, expiresAt future date.
DTO	InviteDealPartyDto	email required, role enum, requiredApproval boolean.
DTO	CreateDealVersionDto	terms required, summary optional, sourceVersionId optional UUID.
DTO	ApprovalDecisionDto	decision enum, optional reason required for rejection or change request.
DTO	SignedUploadUrlDto	bucket enum, fileName, contentType, sizeBytes, dealId or kycSubmissionId.
Guard	SupabaseJwtAuthGuard	Validates Bearer JWT and attaches user identity.
Guard	ResourceAccessGuard	Confirms creator, participant, admin, or service account access.
Guard	AdminRoleGuard	Allows admin and super admin operations only.
Guard	SubscriptionGuard	Enforces trial and paid feature access.

11.5 Standard API Response and Error Format

```
{
  "data": {},
  "meta": {
    "requestId": "uuid",
    "timestamp": "ISO-8601"
  }
}

{
  "error": {
    "code": "DEAL_LOCKED",
    "message": "Locked versions cannot be edited.",
    "details": {},
    "requestId": "uuid"
  }
}
```

Chapter 12

Data Model Specification

12.1 Core Tables

Core tables are profiles, companies, company_members, kyc_submissions, trust_counters, deals, deal_parties, deal_versions, deal_approvals, deal_files, messages, notifications, subscriptions, audit_logs, admin_actions, and reports.

12.2 Relationships

A profile belongs to an auth user. A company has many members. A deal has many parties, versions, files, and messages. A version has many approvals. A user has trust counters and notifications. An admin action belongs to an admin profile.

12.3 Implementation Schema

Table	Required Columns	Keys and Indexes	Ownership and Retention
profiles	id, auth_user_id, display_name, avatar_url, kyc_status, timestamps	pk id, unique auth_user_id, index kyc_status	User-owned; retained while account exists.
companies	id, owner_profile_id, legal_name, status, timestamps	pk id, fk owner_profile_id, index status	Owner-managed; soft delete.
company_members	id, company_id, profile_id, role, status	unique company/profile, indexes company/profile	Company owner managed.
deals	id, creator_profile_id, company_id, title, status, current_version_id, timestamps	pk id, fk creator, index status/creator	Creator and parties; archive not hard delete.
deal_parties	id, deal_id, profile_id, email, party_status, required_approval	unique deal/email, indexes deal/profile/status	Deal creator and invited party.

deal_versions	id, deal_id, version_number, status, terms_json, locked_at	unique deal/version, index deal/status	Immutable after locked_at.
deal_approvals	id, version_id, party_id, approval_status, reason, decided_at	unique version/party, index approval status	Required party-owned decision.
deal_files	id, deal_id, version_id, storage_bucket, storage_path, file_type, visibility	index deal/version	Private; retained with deal history.
messages	id, deal_id, sender_profile_id, body, message_status, created_at	index deal/created time	Participant-only access.
notifications	id, profile_id, notification_type, payload_json, read_at, created_at	index profile/read state	User-owned; retention policy by age.
kyc_submissions	id, profile_id, status, provider_reference, rejection_reason, review timestamps	index profile/status	Private; admin/back-end only.
audit_logs	id, actor_profile_id, action_type, resource_type, resource_id, metadata_json, created_at	index action/resource/time	Immutable platform record.

12.4 Migration Plan

Migrations must be numbered and named by intent, for example `202606190001_initial_schema.sql`. The first migration creates enums and tables, the second creates indexes, the third enables RLS, the fourth creates storage buckets and policies, and seed scripts create development-only sample users, deals, parties, and approvals. Rollbacks must be documented before production migration execution.

Chapter 13

Security and Access Control Specification

13.1 Rules

JWT validation, RBAC, resource-based access control, admin authorization, RLS, private storage policies, KYC document privacy, audit logging, service role restrictions, secret handling, password reset security, rate limiting, input validation, CORS, and error handling are required.

13.2 Access Controls

- Users can only access deals where they are creator or participant.
- Users can only approve versions for deals where they are required parties.
- Users cannot approve on behalf of another user.
- Users cannot edit locked versions.
- Admins can review users and KYC, but actions must be audited.
- Private files are never public.

13.3 RLS and Storage Policy Specification

Resource	Read Policy	Write Policy
<code>profiles</code>	Authenticated user can read own profile; admins can read reviewed profiles.	User can update safe profile fields; KYC and trust fields backend-only.
<code>deals</code>	Creator or participant can read.	Creator can create; lifecycle updates through backend service role only.
<code>deal_parties</code>	Deal participants can read party list.	Creator can invite through API; direct client insert disabled.
<code>deal_versions</code>	Deal participants can read versions.	Draft creation through API; locked versions never update.
<code>deal_approvals</code>	Deal participants can read approval state.	Required party can approve only for own party row through API.

<code>deal_files</code>	Deal participants can read meta-data.	Upload metadata through API after signed URL authorization.
<code>messages</code>	Deal participants can read.	Participants can create through API; moderation can hide.
<code>kyc_submissions</code>	User can read own status summary; admins can read queue.	Submission through API; review admin-only.
<code>audit_logs</code>	Admin-only.	Insert backend-only; update and delete disabled.

Storage buckets are private. The `avatars` bucket may expose signed read URLs for profile display. The `deal-files` bucket is accessible only to authorized deal participants through signed URLs. The `kyc-documents` bucket is accessible only to backend and authorized admin workflows.

13.4 Environment Variable Catalog

Variable	Scope	Purpose
<code>API_PORT</code>	API	Local API port, default 3001.
<code>DATABASE_URL</code>	API	Supabase PostgreSQL connection string.
<code>SUPABASE_URL</code>	API/Admin/Mobile	Supabase project URL.
<code>SUPABASE_ANON_KEY</code>	Client	Public anon key for auth session lifecycle only.
<code>SUPABASE_SERVICE_ROLE_KEY</code>	API	Backend privileged operations; never exposed to clients.
<code>SUPABASE_JWT_SECRET</code>	API	JWT verification when required by auth strategy.
<code>STRIPE_SECRET_KEY</code>	API	Stripe checkout and subscription operations.
<code>STRIPE_WEBHOOK_SECRET</code>	API	Stripe webhook signature verification.
<code>KYC_PROVIDER_API_KEY</code>	API	External identity provider integration.
<code>NEXT_PUBLIC_API_BASE_URL</code>	Admin	Browser-visible NestJS API base URL.
<code>IDEAL_API_BASE_URL</code>	Mobile	Flutter compile-time API base URL.

Chapter 14

API and Integration Specification

14.1 Base Path

All planned REST endpoints use `/api/v1`.

14.2 Endpoint Examples

Method	Path	Purpose
POST	<code>/api/v1/deals</code>	Create a draft deal.
GET	<code>/api/v1/deals</code>	List authorized deals.
GET	<code>/api/v1/deals/:dealId</code>	Read one authorized deal.
POST	<code>/api/v1/deals/:dealId/parties</code>	Invite a deal party.
POST	<code>/api/v1/deals/:dealId/versions</code>	Create a version.
POST	<code>/api/v1/deal-versions/:versionId/submit</code>	Submit for approval.
POST	<code>/api/v1/deal-versions/:versionId/approve</code>	Approve version.
POST	<code>/api/v1/deal-versions/:versionId/reject</code>	Reject version.
POST	<code>/api/v1/deal-versions/:versionId/request-changes</code>	Request changes.
POST	<code>/api/v1/files/signed-upload-url</code>	Create signed upload URL.
GET	<code>/api/v1/notifications</code>	List notifications.
POST	<code>/api/v1/subscriptions/checkout</code>	Start Stripe checkout.
POST	<code>/api/v1/admin/kyc/:submissionId/approve</code>	Approve KYC.

14.3 Implementation API Catalog

Method	Path	DTO	Authorization	Audit
--------	------	-----	---------------	-------

GET	/api/v1/configuration/status	none	public in local/staging; protected in production if required	no
GET	/api/v1/health	none	public	no
POST	/api/v1/deals	CreateDealDto	registered or verified user; subscription checked	yes
GET	/api/v1/deals	DealListQueryDto	creator or participant results only	no
GET	/api/v1/deals/:dealId	DealParamsDto	creator, participant, or admin	no
POST	/api/v1/deals/:dealId/parties	InviteDealPartyDto	creator or authorized company owner	yes
POST	/api/v1/deals/:dealId/versions	CreateDealVersionDto	creator or authorized editor; locked version cannot update	yes
POST	/api/v1/deal-versions/:versionId/submit	VersionParamsDto	creator or authorized editor	yes
POST	/api/v1/deal-versions/:versionId/approve	ApprovalDecisionDto	required party only	yes
POST	/api/v1/deal-versions/:versionId/reject	ApprovalDecisionDto	required party only	yes
POST	/api/v1/deal-versions/:versionId/request-changes	ApprovalDecisionDto	participant only	yes
POST	/api/v1/files/signed-upload-url	SignedUploadUrlDto	participant or KYC owner, depending bucket	yes
GET	/api/v1/notifications	NotificationQueryDto	current user only	no
POST	/api/v1/subscriptions/checkout	CheckoutDto	current user or company owner	yes
POST	/api/v1/admin/kyc/:submitId/approve	AdminKycApprovalDto	admin or super admin	yes

14.4 Request and Response Examples

```

POST /api/v1/deals
{
  "title": "Office renovation agreement",
  "description": "Milestone-based renovation work",
  "currency": "TND"
}

201 Created

```

```
{
  "data": {
    "id": "uuid",
    "status": "draft",
    "currentVersionId": null
  }
}
```

```
POST /api/v1/deal-versions/{versionId}/approve
{
  "decision": "approved"
}

200 OK
{
  "data": {
    "versionId": "uuid",
    "approvalStatus": "approved",
    "dealStatus": "locked"
  }
}
```

Pagination uses `limit`, `cursor`, and `sort`. Filtering uses explicit fields such as `status`, `createdAfter`, and `participantId`. Validation errors return `VALIDATION_ERROR`; authorization errors return `FORBIDDEN`; stale version approvals return `VERSION_CONFLICT`.

Chapter 15

Admin Dashboard Specification

The Next.js admin dashboard is responsible for user management, KYC review, deal activity monitoring, report handling, subscription visibility, trust event inspection, audit log review, admin user management, and platform configuration.

Important Note

The admin dashboard must call NestJS APIs. It must not bypass backend authorization by directly writing to the database.

15.1 Admin Route Catalog

Route	API Dependencies	Responsibility
/admin	/configuration/status, /health	Operational overview and backend connectivity.
/admin/users	/admin/users	Search users, inspect profiles, suspend access.
/admin/users/:id	/admin/users/:id, /admin/audit-logs	User details and action history.
/admin/kyc	/admin/kyc/submissions	KYC queue by status and priority.
/admin/kyc/:id	/admin/kyc/:id/approve, /reject	Review documents, approve, reject, request resubmission.
/admin/deals	/admin/deals	Monitor deal activity without editing locked versions.
/admin/reports	/admin/reports	Review platform reports and moderation state.
/admin/subscriptions	/admin/subscriptions	View subscription state and web-hook history.
/admin/audit-logs	/admin/audit-logs	Filter immutable audit events.
/admin/settings	/admin/settings	Platform configuration for authorized super admins.

15.2 KYC Review Workflow

An admin opens the KYC queue, filters submitted records, reviews private signed document previews issued by the backend, records an approval or rejection reason, and the backend writes an audit event plus a user notification. The dashboard must show loading, empty, error, permission denied, and stale-review states.

15.3 Permission Matrix

Role	Allowed Areas	Restrictions
admin	Users, KYC, reports, deal monitoring, audit read	Cannot manage super admins or bypass backend workflows.
super admin	All admin areas, platform settings, admin management	All actions audited; production changes require approval.
support reviewer	Users, reports, selected audit views	No KYC approval or subscription changes.
finance reviewer	Subscriptions and payment history	No KYC or deal moderation actions.

Chapter 16

Mobile Application Specification

The Flutter app currently has a clean welcome foundation. Target structure is feature-first with core config, network, router, theme, shared widgets, and features for auth, home, profile, KYC, deals, approvals, chat, files, notifications, subscription, and settings.

16.1 Important Refactor Note

Any direct Supabase business logic in Flutter is temporary foundation work. Business actions must move behind NestJS API clients.

16.2 Current Implementation Status

The current mobile application has a starter Flutter shell, project identity assets, a welcome screen, and an API configuration foundation pointing to the NestJS base URL. It does not yet contain final auth screens, route guards, Riverpod feature providers, deal screens, Supabase initialization, or production password reset handling on this branch.

16.3 Target Screen Specification

Screen	Required Data	Main Actions	States
Login	Auth configuration	Sign in, forgot password	loading, invalid credentials, offline.
Register	Profile fields	Create account, accept terms	validation, duplicate email, success.
Reset password	Recovery token	Set new password	token expired, success.
Deals list	Paginated deals	Filter, open, create	loading, empty, error.
Deal details	Deal, parties, current version	Review, invite, edit allowed draft	permission denied, locked, stale.
Invite party	Deal id, email, role	Send invitation	invalid email, duplicate party.
Version history	Versions	Compare, open version	empty, locked marker.
Approval	Version, required parties	Approve, reject, request changes	pending others, conflict, success.
Chat	Deal messages	Send message	participant-only, failed send.

Files	File metadata	Upload, preview, download	signed URL expired, unauthorized.
KYC	Submission status	Submit, resubmit	under review, rejected, approved.
Notifications	Notification list	Mark read, navigate	empty, unread, realtime update.
Settings	Session and preferences	Logout, manage account	confirmation, error.

Chapter 17

File Storage and Media Specification

Storage buckets are avatars, deal-files, and kyc-documents. Files must be private. NestJS issues signed upload and download URLs after authorization checks. File metadata belongs in database tables. Virus scanning is a planned placeholder. Retention rules must distinguish user avatars, deal attachments, and KYC documents.

Chapter 18

Notifications and Realtime Specification

Notifications include deal invitations, approval requests, approvals, rejections, change requests, KYC updates, subscription events, file events, and admin decisions. Realtime updates may use Supabase Realtime, but backend remains the source of notification creation.

Chapter 19

Payments and Subscription Specification

Stripe handles checkout and webhooks. The backend enforces trial contract limits, subscription status, payment failure handling, and paid access. Frontend clients display subscription state but cannot become the source of truth.

Chapter 20

Identity Verification Specification

KYC includes submission, private document upload, provider placeholder, admin review, approval, rejection with reason, resubmission, privacy controls, and retention. KYC documents are private and accessible only to authorized backend or admin workflows.

Chapter 21

Quality Assurance and Testing Plan

Testing includes unit tests, integration tests, API tests, database tests, security tests, RLS tests where direct Supabase access exists, E2E tests, manual QA, acceptance testing, and regression testing.

21.1 Required Scenarios

Registration, login, password reset, deal creation, party invitation, approval, rejection, locked version immutability, unauthorized access denial, file access denial to non-participants, subscription limit enforcement, and admin action audit.

21.2 Testing Matrix

ID	Scenario	Priority	Owner	Automation Target
QA-001	API configuration status responds.	High	Backend	Jest and admin smoke test.
QA-002	Create deal validates request body.	High	Backend	Unit and integration.
QA-003	Unauthorized user cannot read deal.	High	Backend/Database	API and RLS test.
QA-004	Required party approval locks version.	High	Backend	Integration.
QA-005	Locked version cannot be edited.	High	Backend	Unit and integration.
QA-006	Admin KYC decision writes audit log.	High	Admin/Backend	E2E.
QA-007	File signed URL denied to non-participant.	High	Backend/Storage	API test.
QA-008	Mobile displays API configuration.	Medium	Mobile	Widget test.
QA-009	Admin dashboard shows connected/unreachable states.	High	Admin	Component/E2E.

QA-010	PDF builds.	documentation	Medium	Documentation	CI make build.
--------	----------------	---------------	--------	---------------	----------------

21.3 CI Checks

CI must run Flutter analyze, Flutter test, NestJS lint, NestJS test, Next.js lint, Next.js build, Prisma validate/generate after adoption, database migration check, security checks, and PDF documentation build.

Chapter 22

Development Workflow Guidelines

22.1 Branch Naming

feature/mobile-auth, feature/api-deals, feature/admin-kyc,
fix/mobile-password-reset, and docs/product-specification.

22.2 Commit Format

feat(scope): message, fix(scope): message, docs(scope): message,
refactor(scope): message, test(scope): message, and chore(scope): message.

22.3 Daily Update

Yesterday: work completed. Today: planned work. Blocked by: blockers. Need from: required support.

Chapter 23

Delivery Plan and Milestones

Milestone	Deliverables
0 Alignment	LaTeX specification, repo structure, issue board, architecture freeze, MVP approval.
1 Foundation	Flutter, NestJS, Next.js, Supabase database foundation, environment config, CI checks.
2 Identity	Auth flow, profiles, guards, password reset, JWT verification, admin login.
3 Deal Core	Deals, parties, versions, approvals, locked contract logic.
4 Communication	Chat, files, signed URLs, notifications.
5 Trust/KYC/- Payments	KYC, trust counters, subscription enforcement, Stripe.
6 Hardening	Admin dashboard, audit logs, QA, security review, release prep.

23.1 Deliverables by Owner

Owner	Deliverables
Ranime / Mobile owner	Flutter API client, auth screens, profile, deal screens, approval screens, file screens, route guards, widget tests.
Backend owner	NestJS configuration status, auth guard, deal module, party module, version module, approval service, file signed URL service, audit service, OpenAPI contract.
Database owner	Prisma schema, Supabase migrations, RLS policies, storage buckets, seed data, migration checks.
Admin owner	Dashboard shell, live API status panel, admin auth, KYC queue, user management, audit log viewer, admin permission states.
Documentation owner	Product specification, API catalog, database catalog, decision log, revision history, QA matrix.
Project lead	Scope approval, milestone acceptance, release readiness, risk decisions.

23.2 Deployment Environments

Local development uses developer Supabase and local NestJS/Next.js servers. Staging uses isolated Supabase, backend, admin, and Stripe test credentials. Production requires protected branches, approved migrations, monitored backend deployment, signed mobile builds, and rollback plans for API and database releases.

Chapter 24

Risk Register

ID	Description	Probability	Impact	Mitigation
R-001	Architecture drift.	Medium	High	Keep backend ownership documented and reviewed.
R-002	Flutter direct database coupling.	Medium	High	Route business actions through API clients.
R-003	Missing API contracts.	Medium	High	Maintain endpoint catalog.
R-004	Weak RLS or storage policies.	Medium	High	Security review and tests.
R-005	Unclear KYC provider.	High	Medium	Use provider abstraction and placeholder status.
R-006	Overbuilt MVP.	Medium	Medium	Enforce MVP scope statement.

Chapter 25

Decision Log and Open Issues

25.1 Current Decisions

- Flutter is the mobile app framework.
- Next.js is the admin dashboard framework.
- NestJS is the backend API framework.
- Supabase PostgreSQL, Auth, Storage, and Realtime are target platform services.
- Backend owns business logic and audit enforcement.

25.2 Open Issues

Select KYC provider, finalize ORM adoption, define initial Supabase migrations, confirm subscription trial limits, and define CI/CD deployment targets.

25.3 Document Control

Field	Value
Document owner	Documentation owner under project lead authority.
Reviewers	Mobile owner, backend owner, database owner, admin owner, QA representative.
Approvers	Project lead and designated technical lead.
Confidentiality	Internal project team and authorized stakeholders.
Status meaning	Draft for Team Alignment means implementation may begin, but production release requires approval sign-off.
Change control	Any change to API, database, security, or lifecycle rules must update this document and the OpenAPI file in the same pull request.

Backend Implementation Contract

Purpose

This document is the implementation contract for `services/api`. It converts the product specification into backend work that can be reviewed, estimated, tested, and implemented module by module.

Backend Responsibility

NestJS owns business logic, authorization, validation, state transitions, audit logging, signed URL issuance, and integration orchestration. Flutter and Next.js call API endpoints only. The Supabase service role key is API-only and must never be exposed to Flutter, Next.js, logs, or generated client bundles. Sensitive operations write audit records.

Target Folder Structure

```
services/api/src/  
  main.ts  
  app.module.ts  
  config/  
  common/  
  guards/  
  filters/  
  interceptors/  
  decorators/  
  dto/  
  errors/  
  auth/  
  profiles/  
  kyc/  
  deals/  
  deal-parties/  
  deal-versions/  
  approvals/  
  files/  
  messages/  
  notifications/  
  trust/  
  subscriptions/  
  admin/  
  audit/  
  health/
```

prisma/

Common Backend Components

Table 25.2: Backend common components

Component	Contract
<code>SupabaseJwtAuthGuard</code>	Validates Supabase JWT, rejects missing/invalid tokens with <code>UNAUTHORIZED</code> , and attaches the resolved auth user.
<code>ResourceAccessGuard</code>	Checks deal, file, message, and profile resource access before controller methods execute.
<code>AdminRoleGuard</code>	Enforces <code>super_admin</code> , <code>admin</code> , <code>support_reviewer</code> , and <code>finance_reviewer</code> permissions.
<code>SubscriptionGuard</code>	Enforces trial and paid subscription limits before deal creation.
<code>RequestIdInterceptor</code>	Creates or propagates request IDs and adds them to success/error envelopes.
<code>AuditService</code>	Writes append-only audit records for sensitive mutations and admin actions.
<code>HttpExceptionFilter</code>	Converts all exceptions to the standard API error shape.
<code>ValidationPipe</code>	Enforces DTO validation, transform, whitelist, and forbidden non-whitelisted fields.
<code>CurrentUser</code>	Decorator returning authenticated user/profile context.
<code>Roles</code>	Decorator describing role requirements consumed by <code>AdminRoleGuard</code> .

Modules

Table 25.3: Backend module implementation contract

Module	Controllers	Services/DTOs	Authorization and audit	Tests
Auth	<code>AuthController</code> when custom session endpoints are needed	Supabase JWT verification, profile resolver	JWT validation, no service role exposure	Guard tests
Profiles	<code>ProfilesController</code>	<code>ProfilesService</code> , <code>UpdateProfileDto</code>	Own profile safe updates; audit profile changes	Unit/integration
KYC	<code>KycController</code> , <code>AdminKycController</code>	<code>KycService</code> , <code>CreateKycSubmissionDto</code> , <code>AdminKycDecisionDto</code>	Own KYC read/create; admin review; audit review	Unit/integration

Module	Controllers	Services/DTOs	Authorization and audit	Tests
Deals	DealsController	DealsService, CreateDealDto, UpdateDealDto	Creator/participant access; audit create/update/-cancel/archive	Lifecycle tests
Deal Parties	DealPartiesController	DealPartiesService, InviteDealPartyDto	Creator invite/revoke; invited user accept/decline; audit mutations	Access tests
Deal Versions	DealVersionsController	DealVersionsService, CreateDealVersionDto	Unlocked creator creates/submits; audit version events	Version tests
Approvals	ApprovalsController	ApprovalsService, ApprovalDecisionDto	Required party approves exact version; lock when complete; audit all decisions	Conflict tests
Files	FilesController	FilesService, SignedUploadUrlDto, ConfirmUploadDto	Backend-issued signed URLs only; audit confirm/download sensitive files	Storage tests
Messages	MessagesController	MessagesService, CreateMessageDto	Participants only; moderation status honored; audit creates/edits/deletes	Chat tests
Notifications	NotificationsController	NotificationsService	Own notifications only; no cross-user reads	Ownership tests
Trust	TrustService	Trust counters	Backend updates from deal outcomes only	Counter tests
Subscriptions	SubscriptionsController	SubscriptionsService, CheckoutDto	Stripe webhook/service auth; audit subscription events	Webhook tests
Admin	AdminController family	Admin services and DTOs	Admin roles; admin actions and audit logs	Permission tests
Audit	AuditController, AuditService	Audit repository	Append-only; admin read only	Immutability tests
Health	HealthController	HealthService	Public liveness	Smoke tests

Backend Methods

```
DealsService.createDeal(user, dto)
DealsService.listDeals(user, query)
DealsService.getDealById(user, dealId)
DealsService.updateDeal(user, dealId, dto)
DealsService.cancelDeal(user, dealId)
DealsService.archiveDeal(user, dealId)

DealPartiesService.inviteParty(user, dealId, dto)
DealPartiesService.listParties(user, dealId)
DealPartiesService.acceptInvitation(user, partyId)
DealPartiesService.declineInvitation(user, partyId)
DealPartiesService.revokeInvitation(user, partyId)

DealVersionsService.createVersion(user, dealId, dto)
DealVersionsService.listVersions(user, dealId)
DealVersionsService.getVersion(user, versionId)
DealVersionsService.submitVersion(user, versionId)

ApprovalsService.approveVersion(user, versionId, dto)
ApprovalsService.rejectVersion(user, versionId, dto)
ApprovalsService.requestChanges(user, versionId, dto)
ApprovalsService.lockVersionIfComplete(versionId)

FilesService.createSignedUploadUrl(user, dto)
FilesService.confirmUpload(user, dto)
FilesService.listDealFiles(user, dealId)
FilesService.createSignedDownloadUrl(user, fileId)
FilesService.deleteFile(user, fileId)

KycService.getMyKyc(user)
KycService.createSubmission(user, dto)
KycService.createDocumentUploadUrl(user, submissionId, dto)
KycService.approveSubmission(admin, submissionId, dto)
KycService.rejectSubmission(admin, submissionId, dto)

AuditService.record(actionType, actor, resourceType, resourceId, metadata)
```

Authorization Rules

JWT validation uses Supabase signing keys or verified Supabase auth middleware. The backend resolves a profile from the JWT subject before business methods run. Role checks use profile `is_admin` and `admin_role`. Deal access requires creator, linked party profile, matching invited email after acceptance, or authorized admin role. Service role access is available only inside server-side providers. Company permissions require active company membership and role checks.

Error Handling

All responses use the OpenAPI success/error envelope. Required error codes are `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT`,

VERSION_CONFLICT, DEAL_LOCKED, DUPLICATE_PARTY, SUBSCRIPTION_LIMIT, FILE_TOO_LARGE, PAYMENT_CONFIGURATION_MISSING, ADMIN_ONLY, and INTERNAL_ERROR.

Testing Requirements

Each module requires service unit tests, controller integration tests for authorization, validation tests for DTOs, and at least one negative permission test. Approval/version logic requires stale-version and locked-version tests. Admin and audit paths require append-only and permission tests.

Done Criteria

A backend feature is done only when the endpoint exists, DTO validation exists, authorization exists, service logic exists, audit logging exists where required, tests exist, OpenAPI is updated, and the database schema supports the behavior.

Admin Dashboard Specification

Purpose

This document guides `apps/admin` implementation for IDEAL operations, moderation, KYC review, subscriptions, audit visibility, and system status.

Architecture Rule

The Next.js admin dashboard must call NestJS APIs only. It must not write directly to Supabase tables or use the Supabase service role key.

Target Folder Structure

```
apps/admin/  
  app/login/  
  app/admin/page.tsx  
  app/admin/users/  
  app/admin/users/[id]/  
  app/admin/kyc/  
  app/admin/kyc/[id]/  
  app/admin/deals/  
  app/admin/deals/[id]/  
  app/admin/reports/  
  app/admin/subscriptions/  
  app/admin/audit-logs/  
  app/admin/settings/  
  components/  
  lib/api-client.ts  
  lib/auth.ts  
  lib/permissions.ts  
  types/  
  tests/
```

Admin Roles

Table 25.4: Admin roles and permissions

Role	Accessible pages	Allowed actions	Forbidden actions
super_admin	All admin routes	Settings, users, KYC, reports, subscriptions, audit logs	None by role
admin	Users, KYC, deals, reports, audit logs	Approve/reject KYC, moderate reports, inspect deals	Platform settings unless delegated
support_reviewer	Users, KYC queue, reports	Read queues, prepare reviews, resolve allowed reports	Approve KYC, settings, finance data
finance_reviewer	Subscriptions, selected users	Inspect subscriptions and payment state	KYC approval, settings, moderation

Admin Route Catalog

Table 25.5: Admin route catalog

Route	Purpose	API endpoints	UI states/actions
/admin	Overview and API status	/configuration/status, /health	Loading, unreachable, connected, permission denied
/admin/users	User list	/admin/users	Table columns: email, KYC, admin role, status; filters by status/KYC
/admin/users/[id]	User detail	/admin/users/{id}, /admin/users/{id}/status	Suspend/reactivate with confirm dialog
/admin/kyc	KYC queue	/admin/kyc/submissions	Filter submitted/under review; empty and error states
/admin/kyc/[id]	KYC review	/admin/kyc/submissions/{id}, approve/reject endpoints	Signed, preview URL, approve, reject with reason
/admin/deals	Deal monitoring	/admin/deals	Filters by status, creator, date; read-only table
/admin/deals/[id]	Deal inspection	/admin/deals/{id}	Parties, versions, files, messages, audit links
/admin/reports	Moderation reports	/admin/reports, /admin/reports/{id}	Resolve/dismiss reports with reason
/admin/subscriptions	Subscription review	/admin/subscriptions	Finance role access; status filters
/admin/audit-logs	Audit viewer	/admin/audit-logs	Action/resource filters, request ID display

Route	Purpose	API endpoints	UI states/actions
<code>/admin/settings</code>	Platform settings	<code>/admin/settings</code>	Super admin only, confirm every mutation

Required Components

Table 25.6: Admin component catalog

Component	Requirement
AdminLayout	Protected shell with sidebar/topbar and permission fallback.
AdminSidebar	Stable navigation filtered by role.
AdminTopbar	Current user, environment, API status, logout.
StatusBadge, UserStatusBadge, KycStatusBadge	Consistent status rendering.
DataTable	Sort, filter, pagination, empty/error/loading states.
AuditLogTable	Shows action, actor, resource, request ID, timestamp.
ConfirmActionDialog	Required for destructive/sensitive mutations.
PermissionGate	Hides or blocks role-forbidden actions.
ApiStatusPanel	Calls configuration and health endpoints.
KycReviewPanel	Shows KYC submission, signed preview, approve/reject.
ReportReviewPanel	Moderation workflow for reports.

KYC Review Workflow

Admin opens the KYC queue, filters submitted records, opens a submission detail page, requests a backend-issued signed preview URL, approves or rejects, enters a mandatory rejection reason when rejecting, and submits the decision to NestJS. The backend writes an admin action, an audit log, updates the profile KYC status, and creates a user notification.

Admin API Client

The base URL comes from `NEXT_PUBLIC_API_BASE_URL`. The client attaches the bearer token, parses the standard response envelope, parses standard errors, redirects on `UNAUTHORIZED`, renders permission state on `FORBIDDEN`, and displays request IDs for support debugging.

Acceptance Criteria

The admin dashboard is done only when protected routes exist, API status panel exists, KYC queue and user list exist, audit log viewer exists, role permission states exist, loading/empty/error states exist, and tests cover core routes and permissions.

Mobile Integration Contract

Purpose

This document guides `apps/mobile` integration with the NestJS backend and the approved authentication/session model.

Current Status Correction

Current branch evidence shows a Flutter foundation screen with IDEAL branding, main area cards, API configuration constants, endpoint constants, and simple backend readiness probes using Dart `HttpClient`. It does not yet show production auth screens, app routing, Riverpod auth state, Supabase initialization, password reset, deal screens, approval screens, file screens, or widget tests in the current branch. Those items are incomplete and must be implemented or refactored when introduced.

Architecture Rule

Flutter owns UI and local state. NestJS owns business logic. Direct Supabase business logic in Flutter is temporary and should be removed. Flutter may use Supabase Auth only for session lifecycle if approved. No service role keys are allowed in Flutter.

Target Folder Structure

```
apps/mobile/lib/  
  core/config/  
  core/constants/  
  core/errors/  
  core/network/api_client.dart  
  core/network/api_exception.dart  
  core/network/auth_interceptor.dart  
  core/router/  
  core/theme/  
  core/utils/  
  features/auth/data/domain/presentation/  
  features/home/  
  features/profile/  
  features/kyc/  
  features/deals/  
  features/approvals/  
  features/chat/  
  features/files/
```

```

features/notifications/
features/subscription/
features/settings/
shared/widgets/
shared/models/
shared/providers/

```

API Client Requirements

The base URL comes from `IDEAL_API_BASE_URL`. The client attaches bearer tokens, parses the standard success envelope, parses the standard error envelope, handles timeouts, handles offline state, logs requests in debug only, and never stores service role keys.

Flutter Screen Contract

Table 25.7: Mobile screen specification

Screen	Route	Purpose	API endpoints/data	States and validation
Splash	/	Resolve session	Health/config optional	Loading/error
Welcome	/welcome	Brand entry	Configuration status	Connected/unreachable
Login	/login	Sign in	Auth session provider	Email/password validation
Register	/register	Create account	Auth plus profile readiness	Duplicate email/error
Forgot password	/forgot-password	Request reset	Auth provider	Email validation
Reset password	/reset-password	Update password	Auth provider	Token expired/error
Home	/home	Main dashboard	Deals, notifications, profile	Empty/error/offline
Deals list	/deals	Authorized deals	/deals	Pagination/empty
Deal details	/deals/:id	Deal overview	/deals/{id}	Forbidden/not found
Create deal	/deals/new	Draft deal	POST /deals	Title/subscription validation
Edit deal	/deals/:id/edit	Update unlocked deal	PATCH /deals/{id}	Locked error
Invite party	/deals/:id/invite	Add participant	POST /deals/{id}/parties	Email duplicate
Deal parties	/deals/:id/parties	Parties list/actions	Party endpoints	Permission state
Version history	/deals/:id/versions	Version list	Version endpoints	Empty/history
Approval	/versions/:id/approval	Approval/reject	Approval endpoints	Version conflict

Screen	Route	Purpose	API endpoints/data	States and validation
Chat	/deals/:id/chat	Messages	Message endpoints	Non-participant forbidden
Files	/deals/:id/files	Deal files	File endpoints	Signed URL errors
Notifications	/notifications	User alerts	Notification endpoints	Mark read states
Profile	/profile	Profile settings	Profile endpoints	Safe fields only
KYC	/kyc	Identity submission	KYC endpoints	Private file upload
Subscription	/subscription	Plan state	Subscription endpoints	Trial limit/checkout
Settings	/settings	App preferences	Session/profile	Logout/errors

Refactor Requirements

Remove direct deal Supabase calls from Flutter, create a NestJS API client, keep Supabase Auth only if approved, move auth state into clean Riverpod providers when Riverpod is introduced, fix password reset redirects/deep links, add form validation, and add tests.

Done Criteria

A mobile feature is done only when the screen exists, route exists, API client method exists, loading/empty/error/permission states exist, validation exists, errors are displayed, tests exist, and direct business Supabase calls are absent.

QA Acceptance Test Matrix

Table 25.8: QA acceptance matrix

Test ID	Area	Scenario	Preconditions	Steps	Expected Result	Priority	Owner
QA-AUTH-001	Auth	Registration creates profile readiness state	New email	Register	Profile pending readiness	High	Mo
QA-AUTH-002	Auth	Login returns usable session	Existing user	Login	Bearer token works	High	Mo
QA-AUTH-003	Auth	Invalid login shows error	Existing user	Login bad password	Error displayed	High	Mo
QA-AUTH-004	Auth	Password reset sends valid link	Existing email	Request reset	Reset email issued	High	Mo
QA-AUTH-005	Auth	Reset token updates password	Valid token	Submit new password	Password changed	High	Mo
QA-AUTH-006	Auth	Expired reset token shows error	Expired token	Submit password	Token error displayed	Medium	Mo
QA-PROFILE-001	Profile	User reads own profile	Session	GET profile	Own profile returned	High	AP
QA-PROFILE-002	Profile	User updates safe fields	Session	PATCH display name	Safe fields updated	High	AP
QA-PROFILE-003	Profile	User cannot update KYC directly	Session	PATCH KYC status	Forbidden/ignored	High	AP
QA-KYC-001	KYC	User creates KYC submission	Session	POST submission	Submission created	High	AP
QA-KYC-002	KYC	User uploads private KYC document	Submission	Request signed URL	Private upload URL issued	High	AP
QA-KYC-003	KYC	Admin approves KYC	Admin token	Approve	Profile KYC approved	High	Ad
QA-KYC-004	KYC	Admin rejects KYC with reason	Admin token	Reject with reason	Rejected and notified	High	Ad
QA-KYC-005	KYC	User cannot view another KYC document	Two users	Request other document	Forbidden	High	AP

Test ID	Area	Scenario	Preconditions	Steps	Expected Result	Priority	Owner
QA-DEAL-001	Deals	User creates draft deal	Session	POST deal	Draft created	High	AP
QA-DEAL-002	Deals	Creator invites party	Draft deal	POST party	Invitation created	High	AP
QA-DEAL-003	Deals	Invited party accepts	Invitation	Accept	Party accepted	High	AP
QA-DEAL-004	Deals	Unauthorized cannot access deal	Other user	GET deal	Forbidden	High	AP
QA-DEAL-005	Deals	Deal list only authorized deals	Multiple deals	GET deals	Filtered list	High	AP
QA-DEAL-006	Deals	Creator submits version	Draft version	Submit	Pending approval	High	AP
QA-DEAL-007	Deals	Required party approves exact version	Pending version	Approve	Approval recorded	High	AP
QA-DEAL-008	Deals	All approvals lock version	All parties approve	Last approve	Version locked	High	AP
QA-DEAL-009	Deals	Stale approval returns conflict	Old version	Approve stale	VERSION_CONFLICT	High	AP
QA-DEAL-010	Deals	Locked version cannot be edited	Locked deal	Patch version	DEAL_LOCKED	High	AP
QA-DEAL-011	Deals	Post-approval change creates new version	Approved deal	Create change	New version created	Medium	AP
QA-FILE-001	Files	Participant receives upload URL	Deal participant	Request upload URL	Signed URL returned	High	AP
QA-FILE-002	Files	Non-participant cannot upload	Other user	Request URL	Forbidden	High	AP
QA-FILE-003	Files	Participant receives download URL	File exists	Request download	Signed URL returned	High	AP
QA-FILE-004	Files	Non-participant cannot download	Other user	Request download	Forbidden	High	AP
QA-FILE-005	Files	KYC document blocked from participant	Deal participant	Request KYC doc	Forbidden	High	AP
QA-MSG-001	Messages	Participant sends message	Deal participant	POST message	Message sent	Medium	AP
QA-MSG-002	Messages	Non-participant cannot read messages	Other user	GET messages	Forbidden	High	AP
QA-MSG-003	Messages	Hidden message hidden from participant	Hidden msg	GET messages	Hidden absent	Medium	AP
QA-NOTIF-001	Notifications	Invitation creates notification	Invite party	Invite	Notification created	Medium	AP

Test ID	Area	Scenario	Preconditions	Steps	Expected Result	Priority	Owner
QA-NOTIF-002	Notifications	Approval request creates notification	Submit version	Submit	Notification created	Medium	AP
QA-NOTIF-003	Notifications	User marks read	Notification	Mark read	read_at set	Medium	AP
QA-NOTIF-004	Notifications	User cannot read other notification	Other user	GET notification	Forbidden	High	AP
QA-SUB-001	Subscription	Trial allows first five contracts	Trial user	Create five	Allowed	High	AP
QA-SUB-002	Subscription	Sixth contract requires subscription	Trial used	Create sixth	SUBSCRIPTION_LIMIT	High	AP
QA-SUB-003	Subscription	Checkout creates session	Stripe configured	POST checkout	URL returned	High	AP
QA-SUB-004	Subscription	Webhook updates subscription	Stripe event	POST webhook	Status updated	High	AP
QA-SUB-005	Subscription	Frontend cannot override state	Client	PATCH state	Forbidden	High	AP
QA-ADMIN-001	Admin	Admin can access dashboard	Admin token	Open admin	Dashboard visible	High	Ad
QA-ADMIN-002	Admin	Support reviewer cannot approve KYC	Support token	Approve	Forbidden	High	Ad
QA-ADMIN-003	Admin	Admin action writes audit log	Admin action	Mutate	Audit exists	High	AP
QA-ADMIN-004	Admin	Super admin settings access	Super token	Open settings	Settings visible	High	Ad
QA-ADMIN-005	Admin	Finance reviewer subscriptions only	Finance token	Open pages	Subscriptions allowed only	Medium	Ad
QA-SEC-001	Security	Missing token unauthorized	None	Protected call	UNAUTHORIZED	High	AP
QA-SEC-002	Security	Invalid token unauthorized	Bad token	Protected call	UNAUTHORIZED	High	AP
QA-SEC-003	Security	No permission forbidden	Valid token	Forbidden resource	FORBIDDEN	High	AP
QA-SEC-004	Security	Service role never exposed	Client builds	Inspect bundle	No service key	High	All
QA-SEC-005	Security	Audit logs cannot be edited	Admin token	Try update log	Forbidden/no route	High	AP

Test ID	Area	Scenario	Preconditions	Steps	Expected Result	Priority	Owner
QA-SEC-006	Security	Private buckets cannot be listed	User token	List bucket	Forbidden	High	Sto
QA-CI-001	CI	Flutter analyze passes	Dependencies	Run analyze	Pass	High	Mo
QA-CI-002	CI	Flutter tests pass	Tests exist	Run tests	Pass	High	Mo
QA-CI-003	CI	NestJS lint passes	Dependencies	Run lint	Pass	High	AP
QA-CI-004	CI	NestJS tests pass	Tests exist	Run tests	Pass	High	AP
QA-CI-005	CI	Next.js lint passes	Dependencies	Run lint	Pass	High	Ad
QA-CI-006	CI	Next.js build passes	Dependencies	Run build	Pass	High	Ad
QA-CI-007	CI	Prisma validate passes	Prisma installed	Run validate	Pass	High	AP
QA-CI-008	CI	PDF documentation builds	LaTeX installed	Make build	Pass	High	Do

Implementation Status Checklist

Statuses are factual: Done, Partial, Not Started, Blocked, or Needs Review.

Table 25.9: Implementation status checklist

Area	Item	Status	Evidence	Missing Work	Owner	Next Action
Mobile	Flutter project foundation	Done	<code>apps/mobile</code> builds/analyzes	Expand feature modules	Mobile	Add routing
Mobile	Theme	Partial	Material theme seed exists	Full design system	Mobile	Define typography
Mobile	Routing	Not Started	No router package/config	Route table	Mobile	Add routing
Mobile	Auth screens	Not Started	Current screen is welcome/status	Login/register/reset	Mobile	Implement auth features
Mobile	Riverpod auth state	Not Started	Riverpod absent	Providers/session state	Mobile	Add Riverpod needed
Mobile	Supabase initialization	Not Started	No Supabase client in branch	Auth lifecycle decision	Mobile/API	Decide on integration
Mobile	Password reset	Not Started	No reset screen	Deep link handling	Mobile	Design flow
Mobile	API client for NestJS	Partial	Basic <code>HttpClient</code> probe exists	Auth, errors, interceptors	Mobile	Implement contract client
Mobile	Deal screens	Not Started	No feature screens	List/detail/create	Mobile	Create feature
Mobile	Approval screens	Not Started	No approval UI	Version approval flow	Mobile	Create feature
Mobile	File screens	Not Started	No file UI	Signed upload/download	Mobile	Create feature
Mobile	Tests	Not Started	No added widget tests	Widget and integration tests	Mobile	Add tests
Backend	NestJS project foundation	Done	<code>services/api</code> compiles	More modules logic	API	Continue modules
Backend	Config module	Partial	Environment status endpoint	Typed config validation	API	Add config service
Backend	Health endpoint	Done	<code>/api/v1/health</code> exists	None	API	Keep tests
Backend	Auth guard	Not Started	No guard implementation	JWT validation	API	Implement guard

Area	Item	Status	Evidence	Missing Work	Owner	Next Action
Backend	Profile module	Not Started	Contract only	Controller/service/APIs	API	Implement profile feature
Backend	Deal module	Partial	Foundation endpoint only	Real CRUD/services	API	Implement deal service
Backend	Party module	Partial	Contract/schema only	Controllers/services	API	Implement party service
Backend	Version module	Partial	Foundation endpoint only	Version business logic	API	Implement version service
Backend	Approval module	Partial	Foundation endpoint only	Approval/lock logic	API	Implement approval service
Backend	Files module	Partial	Foundation endpoint only	Signed URL integration	API	Implement files service
Backend	Audit module	Partial	Module boundary only	Append-only service	API	Implement audit service
Backend	Admin module	Partial	Foundation endpoint only	Admin routes/-guards	API	Implement admin API
Backend	Tests	Partial	Starter tests pass	Module tests missing	API	Add tests for contract
Backend	OpenAPI alignment	Partial	Contract file exists	Keep synced with implementation	API	Validate CI
Admin	Next.js foundation	Done	App builds	Protected admin routes	Admin	Add routes to tree
Admin	Admin layout	Not Started	Current page is foundation screen	Layout/sidebar/topbar	Admin	Build layout
Admin	API status panel	Partial	Live probe panel exists	Auth-aware panel	Admin	Connect to API
Admin	Auth flow	Not Started	No login/session flow	Protected auth	Admin	Implement auth
Admin	Users page	Not Started	No route	Table/detail/actions	Admin	Create page
Admin	KYC queue	Not Started	No route	Queue/detail/review	Admin	Create feature
Admin	KYC detail	Not Started	No route	Signed preview/decision	Admin	Create page
Admin	Audit logs	Not Started	No route	Viewer/filtering	Admin	Create page
Admin	Permissions	Partial	Contract only	Runtime PermissionGate	Admin	Implement roles
Admin	Tests	Not Started	No added tests	Component/E2E tests	Admin	Add tests
Database	Prisma schema	Done	<code>schema.prisma</code> exists	Validate after Prisma install	API	Run validation

Area	Item	Status	Evidence	Missing Work	Owner	Next Action
Database	Initial migration	Done	Migration 001 exists	Apply to Supabase	API	Run migration
Database	Indexes	Done	Migration 002 exists	Query tuning later	API	Review p
Database	RLS	Partial	Migration 003 baseline exists	Final policy testing	API/Security	Test with keys
Database	Storage buckets	Partial	Migration 004 documents/creates buckets	Environment verification	API	Apply to pabase
Database	Seed data	Done	Development seed exists	Apply locally	API	Seed DB
Documentation	LaTeX PDF	Partial	Builds as draft	Formatting polish	Docs	Fix warn
Documentation	OpenAPI	Done	Contract file exists	Tool validation	API	Validate YAML
Documentation	Backend contract	Done	Contract file exists	Review by backend owner	API	Review
Documentation	Admin contract	Done	Contract file exists	Review by admin owner	Admin	Review
Documentation	Mobile contract	Done	Contract file exists	Review by mobile owner	Mobile	Review
Documentation	QA matrix	Done	Matrix file exists	Automate in CI	QA	Review
Documentation	Decision log	Partial	PDF appendix exists	Track implementation decisions	Team	Update continuously
QA	Backend unit tests	Partial	Starter tests pass	Module tests	API	Add test
QA	Backend integration tests	Not Started	None for real modules	Supertest flows	API	Add test
QA	Mobile widget tests	Not Started	No feature tests	Widget coverage	Mobile	Add test
QA	Admin component tests	Not Started	No component tests	UI state tests	Admin	Add test
QA	E2E tests	Not Started	No E2E harness	Playwright/app flows	QA	Choose ness
QA	Security tests	Not Started	No policy tests	RLS/auth tests	QA/API	Add test
Infrastructure	Supabase migrations	Done	SQL files exist	Apply in environment	API	Run migration
Infrastructure	Storage policies	Partial	Bucket contract exists	Supabase policy verification	API/Security	Test buo
Infrastructure	CI	Not Started	No workflow added in this task	Pipeline setup	Team	Add workflow

Appendix A

Glossary

Term	Definition
Deal	A negotiated agreement workspace.
Version	A specific set of deal terms.
Locked version	Approved official version that cannot be edited.
Required party	Participant whose approval is required.
Audit log	Immutable record of important platform actions.

Appendix B

Status Codes and Enums

B.1 Deal Status

draft, negotiation, pending_approval, approved, locked, changes_requested, rejected, cancelled, archived.

B.2 KYC Status

not_started, submitted, under_review, approved, rejected, resubmission_required.

B.3 Subscription Status

trialing, active, past_due, cancelled, expired.

B.4 Additional Enums

Enum	Values
party_status	invited, accepted, declined, removed, expired
approval_status	pending, approved, rejected, changes_requested, invalidated
notification_type	deal_invitation, approval_requested, approved, rejected, changes_requested, file_uploaded, kyc_updated, subscription_updated, admin_action
file_type	avatar, deal_attachment, kyc_document, generated_contract
file_visibility	private, participant_private, admin_private
company_member_role	owner, manager, member, viewer
admin_role	super_admin, admin, support_reviewer, finance_reviewer
audit_action_type	account_created, kyc_submitted, kyc_approved, deal_created, party_invited, version_created, version_approved, version_locked, file_uploaded, subscription_event, admin_action
report_status	open, under_review, resolved, dismissed
message_status	sent, edited, deleted, hidden
invitation_status	pending, accepted, declined, expired, revoked

Appendix C

Database Table Catalog

Table	Purpose
profiles	User profile metadata linked to auth users.
companies	Company records.
company_members	Company membership and roles.
kyc_submissions	Identity verification workflow records.
trust_counters	Backend-managed trust metrics.
deals	Deal aggregate roots.
deal_parties	Deal participants and required approval state.
deal_versions	Versioned deal terms.
deal_approvals	Per-party version approval records.
deal_files	Private file metadata.
messages	Deal-scoped chat messages.
notifications	User notifications.
subscriptions	Subscription state.
audit_logs	Immutable action records.
admin_actions	Admin decisions and metadata.
reports	User and platform reports.

C.1 Column-Level Implementation Notes

All application tables use UUID primary keys, `created_at`, `updated_at` where mutable, and explicit enum columns for workflow state. Historical workflow records such as approvals, audit logs, locked versions, and admin actions must not be hard-deleted. Mutable aggregates such as deals and profiles may use soft deletion or archive timestamps when product policy requires hiding records from active views.

Appendix D

API Endpoint Catalog

The canonical machine-readable starting point is `docs/api/openapi.yaml`. This appendix summarizes the implementation contract that frontend and backend engineers must follow.

Method	Endpoint	Request	Response	Errors
GET	/configuration/status	none	ConfigurationStatus	500
GET	/health	none	HealthStatus	500
POST	/deals	CreateDealDto	DealSummary	VALIDATION_ERROR, SUBSCRIPTION_LIMIT
GET	/deals	DealListQueryDto	PaginatedDealList	UNAUTHORIZED
GET	/deals/:dealId	DealParamsDto	DealDetails	FORBIDDEN, NOT_FOUND
POST	/deals/:dealId/parties	InviteDealPartyDto	DealParty	DUPLICATE_PARTY
POST	/deals/:dealId/versions	CreateDealVersionDto	DealVersion	DEAL_LOCKED, VERSION_CONFLICT
POST	/deal-versions/:versionId/submit	none	DealVersion	VERSION_CONFLICT
POST	/deal-versions/:versionId/approve	ApprovalDecisionDto	ApprovalResult	FORBIDDEN, VERSION_CONFLICT
POST	/deal-versions/:versionId/reject	ApprovalDecisionDto	ApprovalResult	FORBIDDEN
POST	/deal-versions/:versionId/request-changes	ApprovalDecisionDto	ApprovalResult	FORBIDDEN
POST	/files/signed-upload-url	SignedUploadUrlDto	SignedUrlResponse	FILE_TOO_LARGE, FORBIDDEN
GET	/notifications	NotificationQueryDto	PaginatedNotifications	UNAUTHORIZED
POST	/subscriptions/checkout	CheckoutDto	CheckoutSession	PAYMENT_CONFIGURATION_ERROR
POST	/admin/kyc/:submissionId/approve	AdminKycDecisionDto	KycSubmission	ADMIN_ONLY

Appendix E

Test Scenarios

1. User registration creates profile readiness state.
2. Login returns a usable session.
3. Deal creator invites a party.
4. Party approves the submitted version.
5. Rejection records reason and audit event.
6. Locked version cannot be edited.
7. Non-participant cannot access deal.
8. Non-participant cannot download private file.
9. Subscription limit blocks excess deal creation.
10. Admin action is audited.

Appendix F

Revision History

Table F.1: Document control and revision history

Version	Date	Change	Author	Status
1.0	June 2026	Initial product and technical specification draft.	Team	Draft
1.1	June 2026	Added implementation-grade API catalog, database schema, RLS/storage policies, environment catalog, QA matrix, admin workflows, state machine, OpenAPI starter, and document control.	Team	Draft
1.2	June 2026	Created implementation contracts: OpenAPI, Prisma schema, Supabase migrations, backend contract, admin spec, mobile integration contract, QA matrix, and LaTeX formatting fixes.	Team	Draft